What optimisations are worthwhile for filter, partition, and tokenisation on GPU?

Lilje Henrich Due [1] Troels Henriksen [1] Cosmin Eugen Oancea [1]

[1]Department of Computer Science, University of Copenhagen

August 20th, 2026

Contact: due@di.ku.dk

Motivation

- Primitive Operations
  - Filter
  - Partition
- Non-uniform Data-Parallism
  - Hash Maps
  - Quick Sort
  - Segmented Reduce
- Pointer Structures
  - Expression Evaluation
  - Union-Find
  - List Ranking

Filter written in Futhark

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as
  let offsets = scan (+) 0 (map bool.i64 flags)
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as
  let offsets = scan (+) 0 (map bool.i64 flags)
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as
  let offsets = scan (+) 0 (map bool.i64 flags)
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

$[t, f, t, f, t, f]$

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as                                    [t, f, t, f, t, f]
  let offsets = scan (+) 0 (map bool.i64 flags)           [1, 1, 2, 2, 3, 3]
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as                                        [t, f, t, f, t, f]
  let offsets = scan (+) 0 (map bool.i64 flags)               [1, 1, 2, 2, 3, 3]
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets                                       [0, -1, 1, -1, 2, -1]
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as                                          [t, f, t, f, t, f]
  let offsets = scan (+) 0 (map bool.i64 flags)                 [1, 1, 2, 2, 3, 3]
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets                                         [0, −1, 1, −1, 2, −1]
  let result = scatter (copy as) is as                          [0, 2, 4, 3, 4, 5]
   in take offsets[n - 1] result
```

∗ copied array is actually a scratch array

Filter written in Futhark

Example using an is even predicate: `as = [0, 1, 2, 3, 4, 5]`.

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as                                          [t, f, t, f, t, f]
  let offsets = scan (+) 0 (map bool.i64 flags)                 [1, 1, 2, 2, 3, 3]
  let is = map2 (\f o -> if f then o - 1 else -1)
                flags
                offsets                                         [0, −1, 1, −1, 2, −1]
  let result = scatter (copy as) is as                          [0, 2, 4, 3, 4, 5]
   in take offsets[n - 1] result                                [0, 2, 4]
```

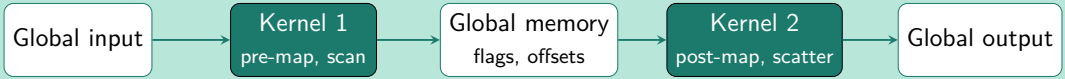
∗ copied array is actually a scratch array

Filter written in Futhark

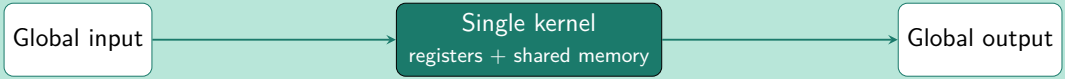
```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as
  let offsets = scan (+) 0 (map bool.i64 flags)
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```

Filter written in Futhark

```
def filter [n] 'a (p:  a -> bool) (as:  [n]a) :  []a =
  let flags = map p as
  let offsets = scan (+) 0 (map bool.i64 flags)
  let is = map2 (\f o -> if f then o - 1 else -1) flags offsets
  let result = scatter (copy as) is as
   in take offsets[n - 1] result
```



Filter written in CUDA



- Streaming Scan.
- Avoid global memory round-trip.
- Not the only optimisation.

Methodology

- Many hand-written CUDA kernels.
- Baseline is a two-kernel implementation mirroring compiler-generated code:
  - Kernel 1: map + scan
  - Kernel 2: map + scatter
- Single-kernel implementations are compared against this baseline.
- Vary optimisations.

Hand-Written CUDA Prototypes of Filter

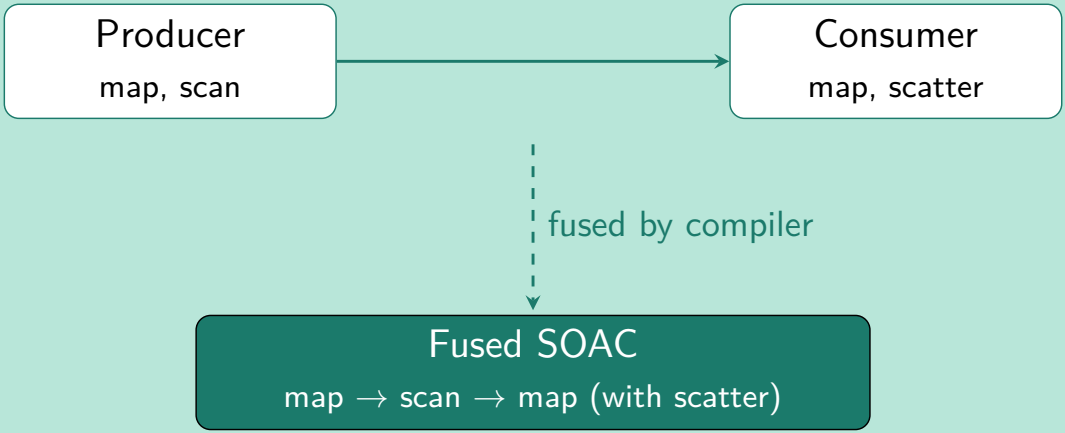
Relative speedup over **2K** baseline (higher is better)

Datasets vary by % of elements written: **Empty** (0%) → **Full** (100%)

Implementation	Full	Dense	Moderate	Sparse	Empty
2K (baseline)	1×	1×	1×	1×	1×
1K	3.5×	3.7×	4×	4.4×	4.9×
B	3.5×	3.7×	4.3×	4.9×	5.7×
CW	3.4×	3.6×	3.8×	4.2×	5.7×
FSMW	3.5×	3.8×	4.2×	5×	5.9×

2K: two-kernel baseline **1K**: single kernel **B**: bitsets in registers **CW**: coalesced writing to global memory **FSMW**: fewer shared memory writes

Fusion of IR



No source language fused primitive — only `map`, `scan`, and `scatter`, combined by fusion.

Scatter as IR

$$
\begin{aligned}
&\texttt{scanscatter}\ f\ (\oplus)\ g\ as\ ds = \\
&\quad \texttt{let}\ (bs, gs)\ =\ (\texttt{unzip} \circ \texttt{map}\ f)\ as \\
&\quad \texttt{let}\ bs'\ =\ \texttt{scan}\ (\oplus)\ bs \\
&\quad \texttt{let}\ (is, vs, ss)\ =\ (\texttt{unzip}_3 \circ \texttt{map}_2\ g)\ bs'\ gs \\
&\quad \texttt{let}\ ds'\ =\ \texttt{scatter}\ ds\ is\ vs \\
&\quad \texttt{in}\ (ds', ss)
\end{aligned}
$$

Accumulators

- Write-only array view; updates accumulate in place.
- Linear types.
- $[](\texttt{acc}\ \tau) \cong \texttt{acc}\ \tau$ — so `map` can take and return accumulators.

$$\texttt{with\_acc} : []\tau \to (\texttt{acc}\ \tau \to \texttt{acc}\ \tau) \to []\tau$$
$$\texttt{update\_acc} : \texttt{acc}\ \tau \to \textbf{int} \to \tau \to \texttt{acc}\ \tau$$

Scatter, expressed via accumulators:

$$\texttt{with\_acc}\ (\lambda \mathit{acc} \to \texttt{map}_2\ (\lambda \mathit{i}\ \mathit{v} \to \texttt{update\_acc}\ \mathit{acc}\ \mathit{i}\ \mathit{v})\ \mathit{is}\ \mathit{vs})\ \mathit{ds}$$

Generalization of IR

$$
\begin{aligned}
&\texttt{maposcanomap}\ f\ (\oplus)\ g\ as = \\
&\quad \texttt{let}\ (bs, gs)\ =\ (\texttt{unzip} \circ \texttt{map}\ f)\ as \\
&\quad \texttt{let}\ bs'\ =\ \texttt{scan}\ (\oplus)\ bs \\
&\quad \texttt{in}\ \texttt{map}_2\ g\ bs'\ gs
\end{aligned}
$$

Generalization of IR

$$
\begin{aligned}
&\texttt{maposcanomap}\ f\ (\oplus)\ g\ as = \\
&\quad \texttt{let}\ (bs, gs)\ =\ (\texttt{unzip} \circ \texttt{map}\ f)\ as \\
&\quad \texttt{let}\ bs'\ =\ \texttt{scan}\ (\oplus)\ bs \\
&\quad \texttt{in}\ \texttt{map}_2\ g\ bs'\ gs
\end{aligned}
$$

$$
\texttt{scan}\ \oplus \equiv \texttt{maposcanomap}\ (\lambda a \to (f\,a, ()))\ (\oplus)\ (\lambda a\ \_ \to a)\ as
$$

Generalization of IR

$$
\begin{aligned}
&\texttt{maposcanomap}\ f\ (\oplus)\ g\ as = \\
&\quad \texttt{let}\ (bs, gs)\ =\ (\texttt{unzip} \circ \texttt{map}\ f)\ as \\
&\quad \texttt{let}\ bs'\ =\ \texttt{scan}\ (\oplus)\ bs \\
&\quad \texttt{in}\ \texttt{map}_2\ g\ bs'\ gs
\end{aligned}
$$

$$\texttt{scan}\ \oplus \equiv \texttt{maposcanomap}\ (\lambda a \to (f\,a, ()))\ (\oplus)\ (\lambda a\ \_ \to a)\ as$$

$$\texttt{scanomap}\ f\ \oplus \equiv \texttt{maposcanomap}\ f\ (\oplus)\ (\lambda a\ b \to (a, b))\ as$$

Results: Filter

Method	Full	Dense	Moderate	Sparse	Empty
Old Futhark	1×	1×	1×	1×	1×
New Futhark	3.5×	3.6×	4×	4.4×	4.5×

Method	Full	Dense	Moderate	Sparse	Empty
New Futhark	1×	1×	1×	1×	1×
CUB	1×	1.1×	1.1×	1.2×	1.2×

Results: Partition

Method	Full	Dense	Moderate	Sparse	Empty
Old Futhark	1×	1×	1×	1×	1×
New Futhark	2.5×	2.6×	2.6×	2.6×	2.3×

Method	Full	Dense	Moderate	Sparse	Empty
New Futhark	1×	1×	1×	1×	1×
CUB	1.4×	1.4×	1.4×	1.4×	1.4×

Results: Tokenizer

Method	Dense	Moderate	Sparse
Old Futhark	1×	1×	1×
New Futhark	1.6×	1.8×	2×

Method	Dense	Moderate	Sparse
New Futhark	1×	1×	1×
2K	1.8×	1.2×	1.1×

- Simulates DFA.
- Determines location and token type.

Conclusion

- Review of hand-written optimisations.
- A simple language extension that generalizes fusion.
- High-level programs become competitative to hand-written code.

Benchmarks

Futhark